

CONSTRAINT IN SQL SERVER

ISA 212 Basisdata 2 , Minggu 3

Mira Musrini Barmawi

Definition

Constraints in SQL Server are predefined rules and restrictions that are enforced in a single column or multiple columns, regarding the values allowed in the columns, to maintain the integrity, accuracy, and reliability of that column's data. In other words, if the inserted data meets the constraint rule, it will be inserted successfully. If the inserted data violates the defined constraint, the insert operation will be aborted.

Definition

- Constraints in SQL Server can be defined at the column level, where it is specified as part of the column definition and will be applied to that column only, or declared independently at the table level. In this case, the constraint rules will be applied to more than one column in the specified table. The constraint can be created within the CREATE TABLE T-SQL command while creating the table or added using ALTER TABLE T-SQL command after creating the table. Adding the constraint after creating the table, the existing data will be checked for the constraint rule before creating that constraint.

THE CONSTRAINT

- There are six main constraints that are commonly used in SQL Server that we will describe deeply with examples within this article and the next one. These constraints are:
- SQL NOT NULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY
- CHECK
- DEFAULT

NOT NULL Constraint in SQL

Assume that we have the below simple CREATE TABLE statement that is used to define the ConstraintDemo1 table. This table contains only two columns, ID and Name. In the ID column definition statement, the SQL NOT NULL column-level constraint is enforced, considering the ID column as a mandatory column that should be provided with a valid SQL NOT NULL value. The case is different for the Name column that can be ignored in the INSERT statement, with the ability to provide it with NULL value. If the null-ability is not specified while defining the column, it will accept the NULL value by default:

EXERCISE

```
Create database SQLShackDemo
```

```
Go
```

```
USE SQLShackDemo
```

```
GO
```

```
CREATE TABLE ConstraintDemo1
```

```
(
```

```
    ID INT NOT NULL,
```

```
    Name VARCHAR(50) NULL
```

```
)
```

EXERCISE

```
INSERT INTO ConstraintDemo1 ([ID],[NAME]) VALUES  
(1,'Ali')
```

```
GO
```

```
INSERT INTO ConstraintDemo1 ([ID]) VALUES (2)
```

```
GO
```

```
INSERT INTO ConstraintDemo1 ([NAME]) VALUES  
('Fadi')
```

```
GO
```

EXERCISE

You will see that the first record will be inserted successfully, as both the ID and Name column's values are provided in the INSERT statement. Providing the ID only in the second INSERT statement will not prevent the insertion process from being completed successfully, due to the fact that the Name column is not mandatory and accepts NULL values. The last insert operation will fail, as we only provide the INSERT statement with a value for the Name column, without providing value for the ID column that is mandatory and cannot be assigned NULL value,

EXERCISE

Messages

(1 row affected)

(1 row affected)

Msg 515, Level 16, State 2, Line 13

Cannot insert the value NULL into column 'ID', table 'SQLShackDemo.dbo.ConstraintDemo1'; column does not allow nulls. INSERT fails.

The statement has been terminated.

EXERCISE

```
ALTER TABLE ConstraintDemo1 ALTER COLUMN [Name]  
VARCHAR(50) NOT NULL
```

You will see that the command will fail, as it will check the existing values of the Name column for NULL values first before creating the constraint, as shown in the error message below:

Messages

```
Msg 515, Level 16, State 2, Line 1  
Cannot insert the value NULL into column 'Name', table 'SQLShackDemo.dbo.ConstraintDemo1'; column does not allow nulls. UPDATE fails.  
The statement has been terminated.
```

EXERCISE

To enforce the NOT NULL Constraints in SQL, we should remove all NULL values of the Name column from the table, using the UPDATE statement below, that replaces the NULL values with empty string:

```
UPDATE ConstraintDemo1 SET [Name]='' WHERE [Name] IS  
NULL
```

If you try to create the Constraints in SQL again, it will be created successfully as shown below :

EXERCISE

```
ALTER TABLE ConstraintDemo1 ALTER COLUMN [Name] VARCHAR(50) NOT NULL
```

100 %

Messages

Commands completed successfully.

UNIQUE Constraints in SQL

The UNIQUE constraint in SQL is used to ensure that no duplicate values will be inserted into a specific column or combination of columns that are participating in the UNIQUE constraint and not part of the PRIMARY KEY. In other words, the index that is automatically created when you define a UNIQUE constraint will guarantee that no two rows in that table can have the same value for the columns participating in that index, with the ability to insert only one unique NULL value to these columns, if the column allows NULL.

UNIQUE Constraints in SQL

- Let us create a small table with two columns, ID and Name. The ID column cannot hold duplicate values due to the UNIQUE constraint specified with the column definition. No restriction defined on the Name column, as in the CREATE TABLE T-SQL statement below:

```
USE SQLShackDemo
GO
CREATE TABLE ConstraintDemo2
(
    ID INT UNIQUE,
    Name VARCHAR(50) NULL
)
```

exercise

```
INSERT INTO ConstraintDemo2 ([ID],[NAME]) VALUES (1,'Ali')
GO
INSERT INTO ConstraintDemo2 ([ID],[NAME]) VALUES (2,'Ali')
GO
INSERT INTO ConstraintDemo2 ([ID],[NAME]) VALUES (NULL,'Adel')
GO
INSERT INTO ConstraintDemo2 ([ID],[NAME]) VALUES (1,'Faris')
```

The first two records will be inserted successfully, with no constraint preventing duplicate values of the Name column. The third record will be inserted successfully too, as the unique ID column allows only one NULL value. The last INSERT statement will fail, as the ID column doesn't allow duplicate values and the provided ID value is already inserted to that column, as shown in the error message below:

EXERCISE

```
Messages

(1 row affected)

(1 row affected)

(1 row affected)
Msg 2627, Level 14, State 1, Line 8
Violation of UNIQUE KEY constraint 'UQ__Constrai__3214EC26B928E528'. Cannot insert duplicate key in object 'dbo.ConstraintDemo2'.
The duplicate key value is (1).
The statement has been terminated.
```

The three inserted rows will be as shown below:

SELECT * FROM ConstraintDemo2

100 %

	ID	Name
1	1	Ali
2	2	Ali
3	NULL	Adel

INFORMATION SCHEMA

```
SELECT CONSTRAINT_NAME,  
       TABLE_SCHEMA ,  
       TABLE_NAME,  
       CONSTRAINT_TYPE  
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS  
WHERE TABLE_NAME='ConstraintDemo2 '
```

Results		Messages		
	CONSTRAINT_NAME	TABLE_SCHEMA	TABLE_NAME	CONSTRAINT_TYPE
1	UQ__Constrai__3214EC26B928E528	dbo	ConstraintDemo2	UNIQUE

Drop constraint unique

Using the constraint name retrieved from the INFORMATION_SCHEMA.TABLE_CONSTRAINTS system object, we can drop the UNIQUE constraint using the ALTER TABLE...DROP CONSTRAINT in SQL T-SQL command below:

```
ALTER TABLE ConstraintDemo2 DROP CONSTRAINT  
[UQ__Constrai__3214EC26B928E528]
```

Insert duplicate values

```
INSERT INTO ConstraintDemo2 ([ID],[Name]) VALUES (1,'Faris')
GO
```

100 %

Messages

(1 row affected)

ADD constraint Unique

Trying to add the UNIQUE constraint in SQL again using the ALTER TABLE...ADD CONSTRAINT T-SQL command below:

```
ALTER TABLE ConstraintDemo2 ADD CONSTRAINT UQ__Constrai  
UNIQUE (ID)
```

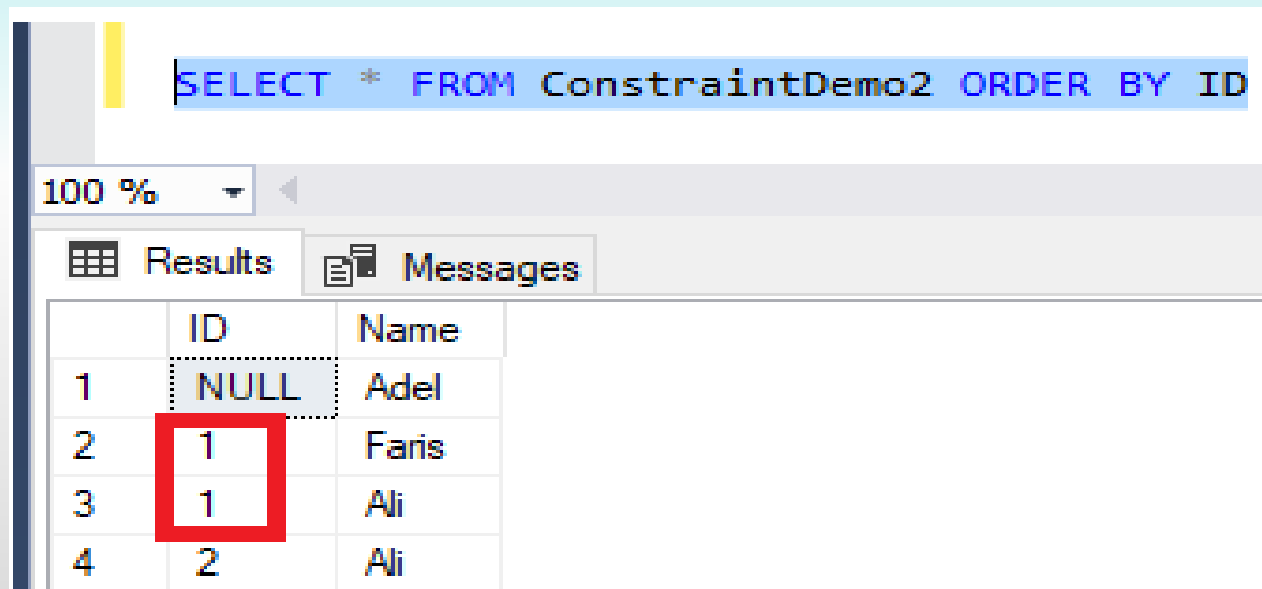
```
GO
```

Messages

```
Msg 1505, Level 16, State 1, Line 5  
The CREATE UNIQUE INDEX statement terminated because a duplicate key was found for the object name 'dbo.ConstraintDemo2' and the index name 'UQ__Constrai'.  
The duplicate key value is (1).  
Msg 1750, Level 16, State 1, Line 5  
Could not create constraint or index. See previous errors.  
The statement has been terminated.
```

Constraint Unique

Checking the inserted data, the duplicate values will be clear as shown below:



```
SELECT * FROM ConstraintDemo2 ORDER BY ID
```

	ID	Name
1	NULL	Adel
2	1	Faris
3	1	Ali
4	2	Ali

PRIMARY KEY Constraint

The PRIMARY KEY constraint consists of one column or multiple columns with values that uniquely identify each row in the table.

The SQL PRIMARY KEY constraint combines between the UNIQUE and SQL NOT NULL constraints, where the column or set of columns that are participating in the PRIMARY KEY cannot accept a NULL value. If the PRIMARY KEY is defined in multiple columns, you can insert duplicate values on each column individually, but the combination values of all PRIMARY KEY columns must be unique.

PRIMARY KEY Constraint

- In addition to providing fast access to the table data, the index that is automatically created, when defining the SQL PRIMARY KEY, will enforce the data uniqueness. The PRIMARY KEY is used mainly to enforce the entity integrity of the table. Entity integrity ensures that each row in the table is a uniquely identifiable entity.

PRIMARY KEY Constraint

- PRIMARY KEY constraint differs from the UNIQUE constraint in that; you can create multiple UNIQUE constraints in a table, with the ability to define only one SQL PRIMARY KEY per each table. Another difference is that the UNIQUE constraint allows for one NULL value, but the PRIMARY KEY does not allow NULL values.

Primary key Constraint

```
USE SQLShackDemo
GO
CREATE TABLE ConstraintDemo3
(
    ID INT PRIMARY KEY,
    Name VARCHAR(50) NULL
)
```

If you try to run the three INSERT statements below:

```
INSERT INTO ConstraintDemo3 ([ID],[NAME]) VALUES (1,'John')
GO
INSERT INTO ConstraintDemo3 ([NAME]) VALUES ('Fadi')
GO
INSERT INTO ConstraintDemo3 ([ID],[NAME]) VALUES (1,'Saeed')
GO
```

Primary key constraint

You will see that the first record will be inserted successfully as both the ID and Name values are valid. The second insert operation will fail, as the ID column is mandatory and cannot be NULL, as the ID column is the SQL PRIMARY KEY. The last INSERT statement will fail too as the provided ID value already exists and the duplicate values are not allowed in the PRIMARY KEY, as shown in the error message below:

```
Messages
(1 row affected)
Msg 515, Level 16, State 2, Line 13
Cannot insert the value NULL into column 'ID', table 'SQLShackDemo.dbo.ConstraintDemo3'; column does not allow nulls. INSERT fails.
The statement has been terminated.
Msg 2627, Level 14, State 1, Line 15
Violation of PRIMARY KEY constraint 'PK__Constrai__3214EC27E0BEB1C4'. Cannot insert duplicate key in object 'dbo.ConstraintDemo3'. The duplicate key value is (1).
The statement has been terminated.
```

```
SELECT * FROM ConstraintDemo3
```

100 %



Results



Messages

	ID	Name
1	1	John

DROP CONSTRAINT PRIMARY KEY

```
SELECT CONSTRAINT_NAME,  
       TABLE_SCHEMA ,  
       TABLE_NAME,  
       CONSTRAINT_TYPE  
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS  
WHERE TABLE_NAME='ConstraintDemo3'
```

Results		Messages		
	CONSTRAINT_NAME	TABLE_SCHEMA	TABLE_NAME	CONSTRAINT_TYPE
1	PK_Constrai_3214EC27E0BEB1C4	dbo	ConstraintDemo3	PRIMARY KEY

With the below result in our example:

The ALTER TABLE...DROP CONSTRAINT T-SQL statement can be used easily to drop the previously defined PRIMARY KEY using the name derived from the previous result:

DROP CONSTRAINT PRIMARY KEY

```
ALTER TABLE ConstraintDemo3
```

```
DROP CONSTRAINT PK__Constrai__3214EC27E0BEB1C4;
```

If you try to execute the previously failed two INSERT statements, you will see that the first record will not be inserted as the ID column does not allow NULL values. The second record will be inserted successfully as there is nothing preventing the duplicate values from being inserted after dropping the SQL PRIMARY KEY, as shown below:

Constraint primary key

```
INSERT INTO ConstraintDemo3 ([NAME]) VALUES ('Fadi')
GO
INSERT INTO ConstraintDemo3 ([ID],[NAME]) VALUES (1,'Saeed')
GO
```

100 %

Messages

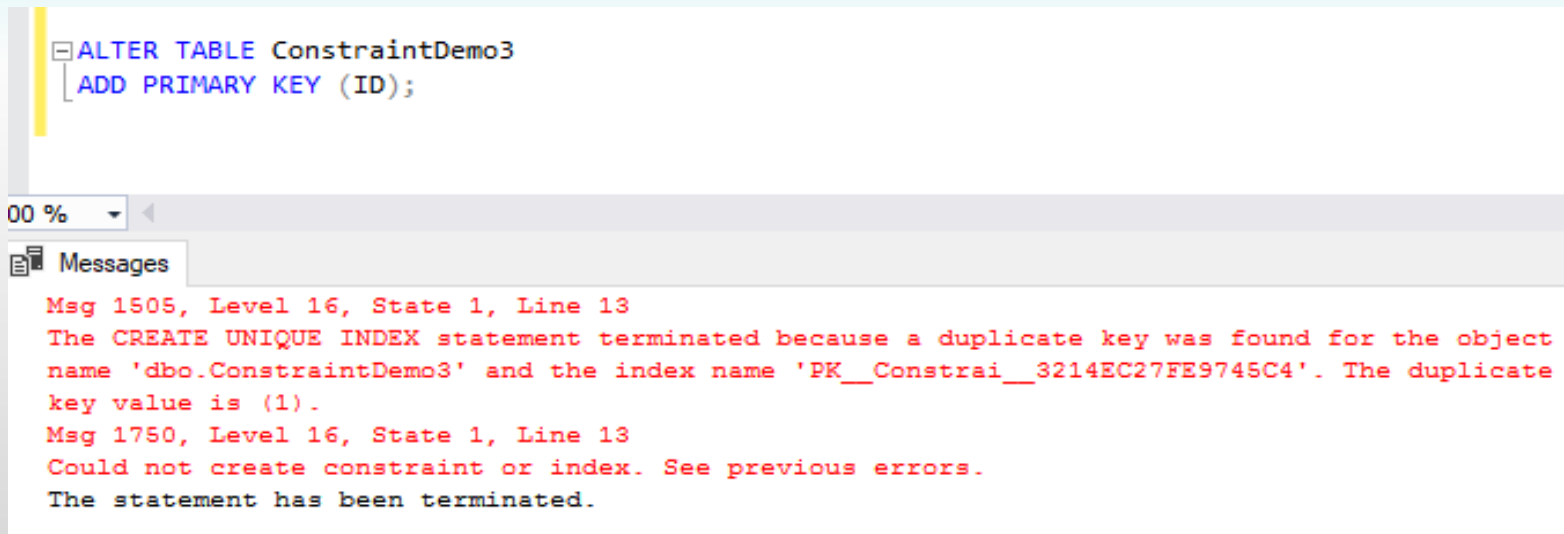
Msg 515, Level 16, State 2, Line 11
Cannot insert the value NULL into column 'ID', table 'SQLShackDemo.dbo.ConstraintDemo3';
column does not allow nulls. INSERT fails.
The statement has been terminated.

(1 row affected)

Constraint primary key

```
ALTER TABLE ConstraintDemo3  
ADD PRIMARY KEY (ID);
```

The operation will fail, as while checking the existing ID values first for any NULL or duplicate values, SQL Server finds a duplicate ID value of 1 as shown in the error message below:

A screenshot of the SQL Server Enterprise Manager interface. The top pane shows a SQL query: `ALTER TABLE ConstraintDemo3 ADD PRIMARY KEY (ID);`. The bottom pane, titled 'Messages', displays two error messages in red text. The first message (Msg 1505) states that the CREATE UNIQUE INDEX statement terminated due to a duplicate key value of (1). The second message (Msg 1750) states that the constraint or index could not be created and the statement was terminated.

```
ALTER TABLE ConstraintDemo3  
ADD PRIMARY KEY (ID);
```

00 %

Messages

Msg 1505, Level 16, State 1, Line 13
The CREATE UNIQUE INDEX statement terminated because a duplicate key was found for the object name 'dbo.ConstraintDemo3' and the index name 'PK__Constrai__3214EC27FE9745C4'. The duplicate key value is (1).

Msg 1750, Level 16, State 1, Line 13
Could not create constraint or index. See previous errors.
The statement has been terminated.

FOREIGN KEY CONSTRAINT

A Foreign Key is a database key that is used to link two tables together. The FOREIGN KEY constraint identifies the relationships between the database tables by referencing a column, or set of columns, in the Child table that contains the foreign key, to the PRIMARY KEY column or set of columns, in the Parent table.

The FOREIGN KEY constraint differs from the PRIMARY KEY constraint in that, you can create only one PRIMARY KEY per each table, with the ability to create multiple FOREIGN KEY constraints in each table by referencing multiple parent table. Another difference is that the FOREIGN KEY allows inserting NULL values if there is no NOT NULL constraint defined on this key, but the PRIMARY KEY does not accept NULLs.

FOREIGN KEY CONSTRAINT

- The FOREIGN KEY constraint provides you also with the ability to control what action will be taken when the referenced value in the parent table is updated or deleted, using the ON UPDATE and ON DELETE clauses. The supported actions that can be taken when deleting or updating the parent table's values include:

FOREIGN KEY CONSTRAINT

- **NO ACTION:** When the ON UPDATE or ON DELETE clauses are set to NO ACTION, the performed update or delete operation in the parent table will fail with an error.
- **CASCADE:** Setting the ON UPDATE or ON DELETE clauses to CASCADE, the same action performed on the referenced values of the parent table will be reflected to the related values in the child table. For example, if the referenced value is deleted in the parent table, all related rows in the child table are also deleted.
- **SET NULL:** With this ON UPDATE and ON DELETE clauses option, if the referenced values in the parent table are deleted or modified, all related values in the child table are set to NULL value.
- **SET DEFAULT:** Using the SET DEFAULT option of the ON UPDATE and ON DELETE clauses specifies that, if the referenced values in the parent table are updated or deleted, the related values in the child table with FOREIGN KEY columns will be set to its default value.

EXERCISE

```
USE SQLShackDemo
GO
CREATE TABLE ConstraintDemoParent
(
    ID INT PRIMARY KEY,
    Name VARCHAR(50) NULL
)
GO
CREATE TABLE ConstraintDemoChild
(
    CID INT PRIMARY KEY,
    ID INT FOREIGN KEY REFERENCES ConstraintDemoParent(ID)
)
```

EXERCISE

```
INSERT INTO ConstraintDemoParent ([ID],[NAME]) VALUES  
(1,'John'), (2,'Mika'), (3,'Sanya')
```

```
GO
```

```
INSERT INTO ConstraintDemoChild (CID,ID) VALUES (1,1)
```

```
GO
```

```
INSERT INTO ConstraintDemoChild (CID,ID) VALUES (2,4)
```

```
GO
```

EXERCISE

The result will show you that the three records are successfully inserted into the parent table. The first record that we tried to insert into the child table is inserted without any error as the ID value of 1 already exists in the parent table. Trying to insert the second record into the child table will fail because the ID value of 4 doesn't exist in the parent table, and due to the FOREIGN KEY constraint, you will not be able to insert an ID value to the child table that doesn't exist in the parent table:

EXERCISE

```
SELECT * FROM ConstraintDemoParent
GO
SELECT * FROM ConstraintDemoChild
```

100 %

Results Messages

	ID	Name
1	1	John
2	2	Mika
3	3	Sanya

	CID	ID
1	1	1

EXERCISE

```
SELECT CONSTRAINT_NAME,  
       TABLE_SCHEMA ,  
       TABLE_NAME,  
       CONSTRAINT_TYPE  
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS  
WHERE TABLE_NAME='ConstraintDemoChild'
```

	CONSTRAINT_NAME	TABLE_SCHEMA	TABLE_NAME	CONSTRAINT_TYPE
1	PK__Constrai__C1F8DC593DEB3D8A	dbo	ConstraintDemoChild	PRIMARY KEY
2	FK__ConstraintDe__ID__0B91BA14	dbo	ConstraintDemoChild	FOREIGN KEY

Drop foreign key constraint

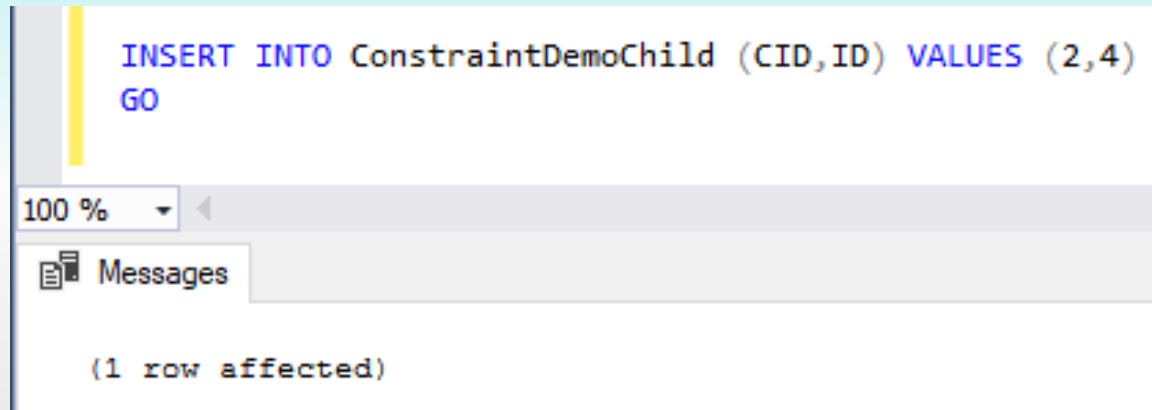
Then we can easily use the previous result to drop the FOREIGN KEY constraint using the following

ALTER TABEL ... DROP CONSTRAINT T-SQL statement:

```
ALTER TABLE ConstraintDemoChild  
DROP CONSTRAINT FK__ConstraintDe__ID__0B91BA14;
```


Drop foreign key constraint

- Trying to insert the second record into the child table again, the insert operation will succeed without any error as shown below:



The screenshot shows a SQL query execution window. The query text is: `INSERT INTO ConstraintDemoChild (CID,ID) VALUES (2,4)` followed by `GO` on a new line. Below the query, there is a progress bar showing 100% completion. A tab labeled 'Messages' is visible, and the output area displays `(1 row affected)`.

```
INSERT INTO ConstraintDemoChild (CID,ID) VALUES (2,4)
GO
```

100 %

Messages

(1 row affected)

Add FOREIGN KEY constraint

```
ALTER TABLE ConstraintDemoChild  
ADD CONSTRAINT FK__ConstraintDe__ID  
FOREIGN KEY (ID) REFERENCES ConstraintDemoParent (ID);
```

The operation will fail, as the ID value of 4 does not exist in the parent table and breaks the referential integrity between the child and parent tables, as seen in the following error message:

Add FOREIGN KEY constraint

Messages

Msg 547, Level 16, State 0, Line 1
The ALTER TABLE statement conflicted with the FOREIGN KEY constraint "FK__ConstraintDe__ID". The conflict occurred in database "SQLShackDemo", table "dbo.ConstraintDemoParent", column 'ID'.

```
UPDATE [dbo].[ConstraintDemoChild] SET ID =2 WHERE ID = 4
```

Now the FOREIGN KEY constraint will be created successfully without any error as shown below:

```
ALTER TABLE ConstraintDemoChild  
ADD CONSTRAINT FK__ConstraintDe__ID  
FOREIGN KEY (ID) REFERENCES ConstraintDemoParent(ID);
```

100 %

Messages

Commands completed successfully.

- <https://www.sqlshack.com/commonly-used-sql-server-constraints-foreign-key-check-default/>

CHECK Constraint

A CHECK constraint is defined on a column or set of columns to limit the range of values, that can be inserted into these columns, using a predefined condition. The CHECK constraint comes into action to evaluate the inserted or modified values, where the value that satisfies the condition will be inserted into the table, otherwise, the insert operation will be discarded. It is allowed to specify multiple CHECK constraints for the same column.

CHECK Constraint

Defining the CHECK constraint condition is somehow similar to writing the WHERE clause of a query, using the different comparison operators, such as AND, OR, BETWEEN, IN, LIKE and IS NULL to write its Boolean expression that will return TRUE, FALSE or UNKNOWN. The CHECK constraint will return UNKNOWN value when a NULL value is present in the condition. The CHECK constraint is used mainly to enforce the domain integrity by limiting the inserted values to the ones that follow the defined values, range or format rules.

Check constraint exercise

Let us create a new simple table that has three columns; the ID column that is considered as the PRIMARY KEY of that table, Name, and Salary.

```
CREATE TABLE ConstraintDemo4
(
    ID INT PRIMARY KEY,
    Name VARCHAR(50) NULL,
    Salary INT CHECK (Salary>0)
)
GO
INSERT INTO ConstraintDemo4 ([ID],[NAME],Salary) VALUES (1,'John',350)
GO
INSERT INTO ConstraintDemo4 ([ID],[NAME],Salary) VALUES (2,'Mike',0)
GO
INSERT INTO ConstraintDemo4 ([ID],[NAME],Salary) VALUES (3,'Nikola',-72)
GO
```

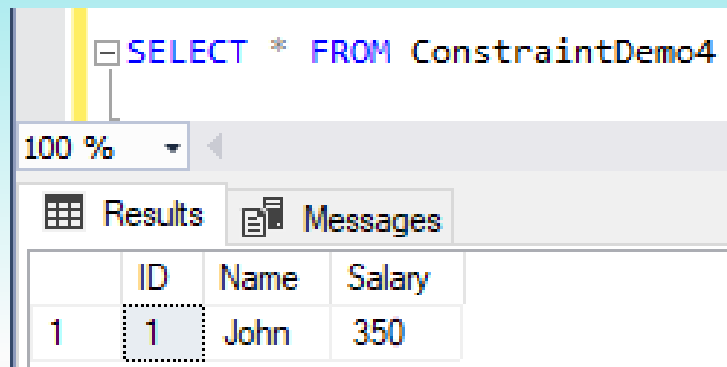
CHECK CONSTRAINT

- It is clear from the generated result that the first record is inserted with no error as the provided Salary value meets the checking condition. The second and third INSERT statements will fail, as the provided Salary values do not meet the CHECK constraint condition due to inserting zero and negative Salary values, as you can see in the following error message:

Messages

```
(1 row affected)
Msg 547, Level 16, State 0, Line 14
The INSERT statement conflicted with the CHECK constraint "CK__Constrain__Salar__0F624AF8". The conflict occurred in database "SQLShackDemo",
table "dbo.ConstraintDemo4", column 'Salary'.
The statement has been terminated.
Msg 547, Level 16, State 0, Line 16
The INSERT statement conflicted with the CHECK constraint "CK__Constrain__Salar__0F624AF8". The conflict occurred in database "SQLShackDemo",
table "dbo.ConstraintDemo4", column 'Salary'.
The statement has been terminated.
```


CHECK CONSTRAINT



The screenshot shows a database query interface. At the top, a SQL query is entered: `SELECT * FROM ConstraintDemo4`. Below the query, there is a zoom level dropdown set to "100 %". Underneath, there are two tabs: "Results" (active) and "Messages". The "Results" tab displays a table with the following data:

	ID	Name	Salary
1	1	John	350

CHECK CONSTRAINT

- We mentioned earlier in this article that the CHECK constraint allows inserting NULL values if the participated columns allow NULL, and the NULL values will be evaluated as UNKNOWN without throwing any error.

```
INSERT INTO ConstraintDemo4 ([ID],[NAME],Salary) VALUES (4,'Philip',NULL)  
GO
```

100 %

 Messages

(1 row affected)

CHECK CONSTRAINT

- If you review the CHECK constraint definition in the previous CREATE TABLE statement, you will see that we have not mentioned the name of the defined constraint. In this case, SQL Server will assign a unique name for that constraint that can be shown by querying the INFORMATION_SCHEMA.TABLE_CONSTRAINTS system view for the ConstraintDemo4 table.

```
SELECT CONSTRAINT_NAME,  
       TABLE_SCHEMA ,  
       TABLE_NAME,  
       CONSTRAINT_TYPE  
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS  
WHERE TABLE_NAME='ConstraintDemo4'
```

DROP CHECK CONSTRAINT

Results Messages				
	CONSTRAINT_NAME	TABLE_SCHEMA	TABLE_NAME	CONSTRAINT_TYPE
1	PK__Constrai__3214EC27E70B4A89	dbo	ConstraintDemo4	PRIMARY KEY
2	CK__Constrain__Salar__0F624AF8	dbo	ConstraintDemo4	CHECK

The CHECK constraint can be dropped using the ALTER TABLE T-SQL statement. Using the CHCEK constraint name we got previously,

```
ALTER TABLE ConstraintDemo4  
DROP CONSTRAINT CK__Constrain__Salar__0F624AF8;
```

DROP CHECK CONSTRAINT

```
INSERT INTO ConstraintDemo4 ([ID],[NAME],Salary) VALUES (2,'Mike',0)
GO
INSERT INTO ConstraintDemo4 ([ID],[NAME],Salary) VALUES (3,'Nikola',-72)
GO
```

100 %

Messages

(1 row affected)

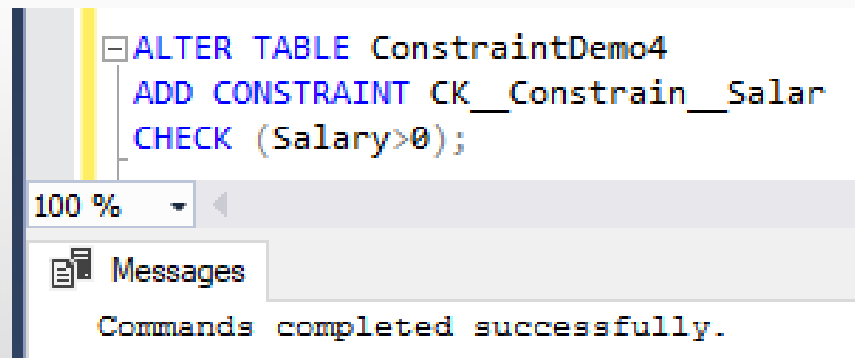
(1 row affected)

ADD CHECK CONSTRAINT

- To be able to define the CHECK constraint again, we should modify or delete the data that prevents the constraint from being created. We will try modifying the Salary amount of the two records that break the constraint condition to a valid positive value, as in the following UPDATE statement

UPDATE ConstraintDemo4 SET Salary =300 WHERE ID IN (2,3)

- Now, the CHECK constraint can be defined with no issue as shown below:



The screenshot shows a SQL Server Enterprise Manager interface. The top pane displays the following SQL command: `ALTER TABLE ConstraintDemo4 ADD CONSTRAINT CK__Constrain__Salar CHECK (Salary>0);`. Below the command pane, a 'Messages' pane shows the status 'Commands completed successfully.'.

```
ALTER TABLE ConstraintDemo4
ADD CONSTRAINT CK__Constrain__Salar
CHECK (Salary>0);
```

100 %

Messages

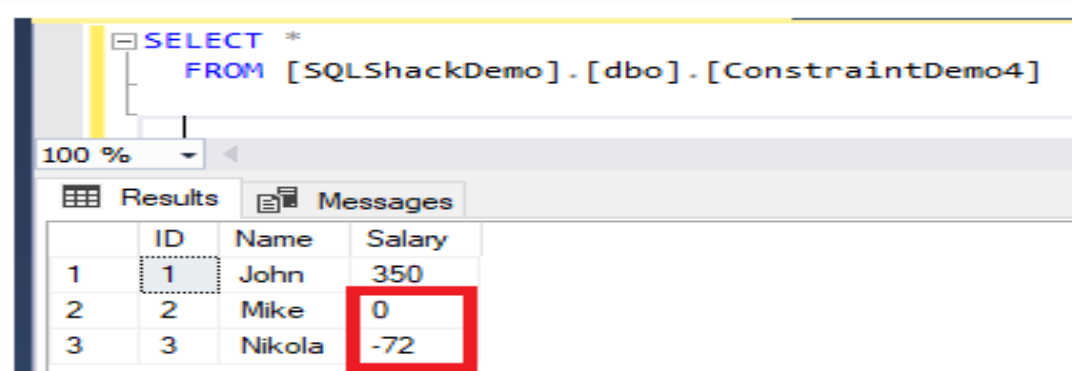
Commands completed successfully.

DISABLE CHECK CONSTRAINT

- SQL Server allows you to disable the CHECK constraint for special cases, such as inserting huge amount of data without caring about meeting the constraint condition for testing purposes, or due to changing in the business logic. T

**ALTER TABLE ConstraintDemo4 NOCHECK CONSTRAINT
CK__Constrain__Salar**

- Trying to insert a negative Salary value will not fail, as there is no validation check performed on that column, as shown below:



```
SELECT *
FROM [SQLShackDemo].[dbo].[ConstraintDemo4]
```

	ID	Name	Salary
1	1	John	350
2	2	Mike	0
3	3	Nikola	-72

ENABLE CHECK CONSTRAINT

If you try to drop the CHECK constraint and create it again using the NOCHECK option, you will see that the constraint is created without checking the existing data, as shown below:

```
ALTER TABLE ConstraintDemo4 WITH NOCHECK
ADD CONSTRAINT CK__Constrain__Salary
CHECK (Salary>0)
```

100 %

Messages

Commands completed successfully.

```
INSERT INTO ConstraintDemo4 ([ID],[NAME],Salary) VALUES (3,'Zaid',-10)
GO
```

100 %

Messages

Msg 2627, Level 14, State 1, Line 7
Violation of PRIMARY KEY constraint 'PK__Constrai__3214EC27E70B4A89'. Cannot insert duplicate key in object 'dbo.ConstraintDemo4'. The duplicate key value is (3).
The statement has been terminated.

DEFAULT Constraint

A DEFAULT constraint is used to provide a default column value for the inserted rows if no value is specified for that column in the INSERT statement. The Default constraint helps in maintaining the domain integrity by providing proper values for the column, in case the user does not provide a value for it. The default value can be a constant value, a system function value or NULL.

Example:

```
CREATE TABLE ConstraintDemo5
(
    ID INT PRIMARY KEY,
    Name VARCHAR(50) NULL,
    EmployeeDate DATETIME NOT NULL DEFAULT GETDATE()
)
GO
```

DEFAULT Constraint

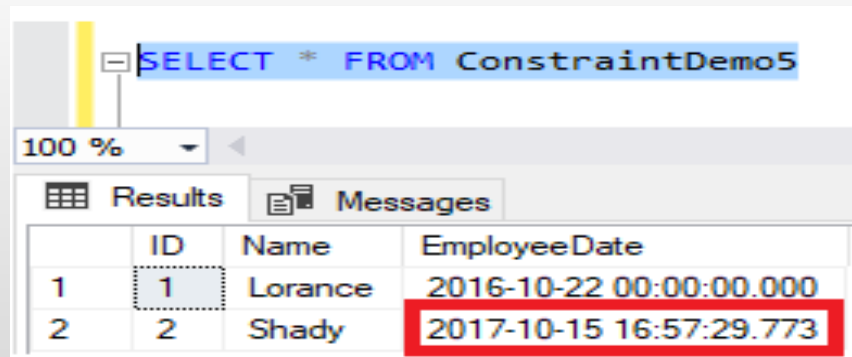
If we execute the two INSERT statements below:

```
INSERT INTO ConstraintDemo5 ([ID],[NAME],EmployeeDate) VALUES  
(1,'Lorance','2016/10/22')
```

GO

```
INSERT INTO ConstraintDemo5 ([ID],[NAME]) VALUES (2,'Shady')
```

GO



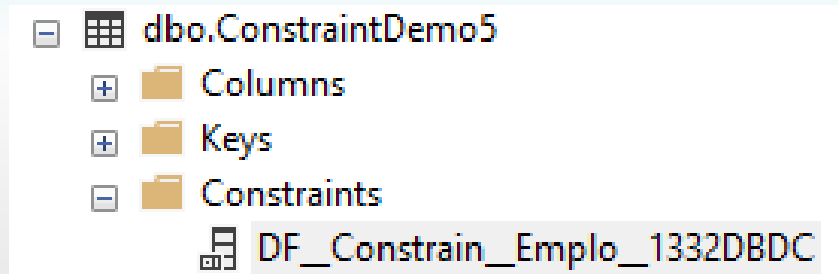
100 %

Results Messages

	ID	Name	EmployeeDate
1	1	Lorance	2016-10-22 00:00:00.000
2	2	Shady	2017-10-15 16:57:29.773

DROP DEFAULT Constraint

Expanding the Constraints node under the current table will show us the created DEFAULT constraint name, recalling that SQL Server will assign a unique name for it if we do not provide a name for it, as shown below:



DROP DEFAULT Constraint

- The DEFAULT constraint can be easily dropped using the ALTER TABLE ... DROP CONSTRAINT T-SQL command below:

```
ALTER TABLE ConstraintDemo5
```

```
DROP CONSTRAINT DF__Constrain__Emplo__1332DBDC;
```

ADD DEFAULT Constraint

- And created using the ALTER TABLE ...ADD CONSTRAINT T-SQL command below:

```
ALTER TABLE ConstraintDemo5
```

```
Add Constraint DF__Constrain__Emplo  DEFAULT (GETDATE())  
FOR EmployeeDate
```